

models (TLMs) and register transfer level (RTL) models, using five basic engines.

1. Virtual prototyping based on TLM models
2. RTL simulation
3. Emulation
4. FPGA based prototyping
5. Bringing up

FPGA basic engines

FPGA based emulators are typically weaker with respect to debug efficiency and turnaround time, making these less reactive and similar to FPGA based prototypes. The downside to standard FPGA based prototyping is capacity limitations as well as longer bring-up due to the changes that have to be made to map the RTL to the FPGAs. So only the efficient combination of the four engines, namely, emulation, virtual prototyping, RTL simulation and FPGA based prototyping, provides a complete solution.

Emulation. Emulation extends verification to the full chip and at the chip-in-system level by enabling connections to real system environments like PCI, USB and Ethernet. The main advantage of processor based emulation is fast turnaround time for bring-up, which makes it ideal for the project phase in which RTL is not quite mature yet.

In addition, it allows multi-user access and excellent hardware debug insight in the context of real software that can be executed at MHz speeds, resulting in very efficient hardware/software debug cycles. Standard software debuggers can be attached using JTAG adaptors or virtual connections.

Virtual prototyping. Virtual prototyping can enable software development as early as a fortnight after the specification is available. It does not allow detailed hardware debug, which is the initial strength of RTL simulation. Used initially for RTL development, IP integration and design verification, RTL simulation can extend to the complexity of sub-systems and is a sign-off criterion for gate-level simulation and timing sign-off. It allows the

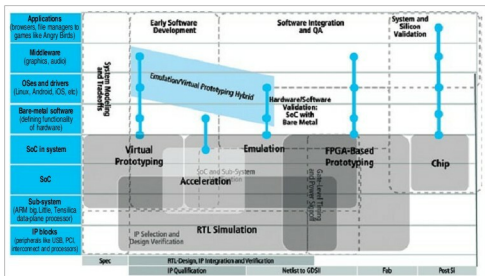


Fig. 2: Four basic engine sweet spots and scopes

fastest turnaround time for new RTLs, offers excellent hardware debug but is typically too slow to execute substantial amounts of software. To better extend to sub-systems and the full SoC, verification acceleration moves the device under test into hardware and can allow enough speed up for bare-metal software development.

The different engine sweet spots as an overlay on the main user tasks are shown in Fig. 3.

Techniques

Emulation and RTL simulation are combined to achieve simulation acceleration. Emulation and TLMs with fast models from ARM are combined to accelerate operating system (OS) bring-up and software-driven testing. Emulation and FPGA based prototyping are combined to combine the speed of bring-up for new portions of the design in emulation with the speed of execution for stable portions of the design in FPGA based prototyping.

Hardware assisted verification

Starting at RTL level, the ultimate goal is to make the model execute faster than it could in a software simulator. Another issue is that, software simulation slows down significantly

Gate-level simulation	- Speed: 1Hz-10Hz - Debug/visibility: Full, large dumps - Boot, clock/reset, basic functionality
Simulation (RTL sim)	- Speed: 1KHz-10kKHz - Debug/visibility: Full - UVM random, feature testing
Emulation (Palladium)	- Speed: 1MHz-3MHz - Debug/visibility: Full but for limited time - System scenario, performance
Prototype (FPGA)	- Speed: 5MHz-50MHz - Debug/visibility: Very limited (100K-1K signals) - FW development, silicon-like testing

Fig. 3: An example of how multiple platforms are used in conjunction

when it exceeds the physical memory of the computer, so there is a capacity issue that plays here as well.

There are two general ways to solve the problem, referred to as direct and indirect implementations.

In the case of direct implementation, while compiling a model into an FPGA, an actual implementation of that model in hardware is created. It may not be the same as the one used in an SoC but it is an implementation nonetheless. Mapping emulates the function of the intended hardware. This technique is generally referred to as emulators, because these are directly executing an implementation of the model.

In indirect implementation, mechanisms are devised that allow effects of concurrency to be evaluated even though the simulator is actually incapable of doing more than one thing at a time.